Classification and Generation Are Complementary Formulations for Code Execution Verification: An Empirical Characterization

Anonymous

Abstract

Execution-free verification of candidate patches is central to test-time scaling for automated code repair, yet the choice between predicting a binary pass/fail label (classification, CLS) and generating the expected execution output (code-world-model, CWM) remains unexplored. We conduct a controlled empirical study comparing these two formulations across three model architectures (Qwen3-4B, Qwen3-8B, DeepSeek-Coder-6.7B) at both function-level and repo-level granularity. Our findings are threefold. **(1) Efficiency equivalence**: CLS produces a single output token yet matches or exceeds CWM accuracy, yielding an 1.9–8.8× latency reduction on GPU (more pronounced at repo-level). **(2) Oracle characterization**: per-test CLS granularity adds diagnostic value only when there are multiple fail-to-pass tests ($T \geq 2$, 28.8% of SWE-bench Verified instances) and regression rates exceed $\sim 10\%$; we derive a practical decision framework for when per-test verification is cost-effective. **(3) Error complementarity**: CLS and CWM exhibit systematically opposite failure modes—CLS false negatives dominate (75.5%) while CWM false positives dominate (83.1%)—enabling an oracle ensemble ceiling of 90.63% (4B) to 93.1% (8B). We release SWE-bench-PerTest, a dataset of 458 instances with per-test labels across 16 candidate patches, and argue that formulation diversity is a promising verification strategy.

1 Introduction

Automated program repair has advanced rapidly through LLM-based agents (Jimenez et al., 2024; Zhang et al., 2024), but the *verification bottleneck* remains: ranking candidate patches requires expensive test execution ($1–3 and minutes per candidate). Execution-free alternatives fall into two paradigms. *Reward models* (Zhang et al., 2025) predict a single trajectory-level correctness score but sacrifice per-test diagnostic information. *Code World Models* (CWM) (Gu et al., 2024) generate expected execution outputs, providing rich signals but suffering from token-budget exhaustion and string-value prediction errors on complex code.

A natural middle ground—predicting a binary PASS/FAIL label per test—has not been studied as a dedicated verification formulation. This paper fills that gap through a controlled comparison of classification (CLS) and generation (CWM) formulations.

Rather than claiming one formulation dominates, our central thesis is that **classification and generation are complementary formulations with predictable, architecture-dependent failure modes**. We support this with three lines of evidence:

1. **Efficiency equivalence.** CLS achieves comparable or higher accuracy with a single output token, reducing inference latency by 1.9× (function-level GPU) to 8.8× (repo-level GPU) relative to CWM.

2. **Oracle characterization.** Per-test granularity is beneficial only under specific data conditions: $T \geq 2$ fail-to-pass tests and regression rates $\geq 10\%$. We provide an empirically grounded decision framework.

3. **Error complementarity.** CLS and CWM errors are systematically asymmetric (CLS: conservative, misses bugs; CWM: noisy generation, produces false alarms). An oracle ensemble achieves 90.63%–93.1%, motivating learned ensemble strategies.

We additionally find that the CLS–CWM accuracy gap is architecture-dependent: the gap is +2.43 pp on Qwen3-8B but only +0.52 pp on DeepSeek-Coder-6.7B, a code-specialized model whose stronger generation pretraining closes the

CWM deficit. This further supports complementarity over dominance.

Our contributions include: (1) the first controlled CLS-vs-CWM comparison for code verification; (2) SWE-BENCH-PERTEST, a per-test labeled dataset; (3) an empirical characterization of when each formulation is preferable; and (4) evidence that formulation diversity offers a path to stronger verification.

2 Related Work

Execution-free code verification. SWE-RewardModel (Zhang et al., 2025) trains a reward model on trajectory-level correctness signals to rank patches without execution. CodeRM (Shi et al., 2024) and outcome reward models for code generation (Li, 2025) similarly predict correctness at the candidate level. These methods provide no per-test diagnostic signal.

Code World Models. CRUXEval (Gu et al., 2024) introduced benchmarks for predicting code execution outputs. Self-Execution Simulation (Ni et al., 2024) extends this to multi-step execution traces. These approaches generate expected outputs, providing rich diagnostic information but requiring many output tokens and suffering from string-value prediction errors.

Per-test verification. To our knowledge, no prior work frames per-test pass/fail prediction as a dedicated classification task for patch verification. Our work bridges execution-free verification and per-test diagnostics.

3 Method

3.1 Problem Setting

Given a candidate patch p for a software issue, and a set of T associated tests $\{t_1, \ldots, t_T\}$, per-test verification predicts $\hat{y}_i \in \{\text{PASS}, \text{FAIL}\}$ for each (p, t_i) pair *without executing the code*. This contrasts with trajectory-level verification (predicting a single label for the entire patch) and CWM (predicting the execution output string for each test).

3.2 Classification Formulation (CLS)

The input is a concatenation of the patch diff, test source code, and a task-specific prompt. The model is fine-tuned to produce a single token: `pass` or `fail`. Output length is exactly 1–2 tokens, independent of test complexity.

Metric	Value
Instances (successful)	458
Candidates per instance (mean)	16
Total (code, test) pairs	115,872
F2P tests per instance (mean / median)	2.1 / 1
P2P tests per instance (mean)	13.7
$T = 1$ instances	71.2%
$T \geq 2$ instances	28.8%
Overall pass rate	73%
P2P regression rate	0.8%

Table 1: SWE-bench-PerTest dataset statistics.

3.3 Generation Formulation (CWM)

The input is the same patch–test pair, but the model generates the expected execution output (e.g., assertion results, stdout). The generated output is then compared to the ground truth to derive a pass/fail label. Output length varies from ∼6 tokens (function-level) to ∼45 tokens mean / 180 tokens P95 (repo-level).

3.4 Training Setup

All models are fine-tuned with LoRA (rank=16, α=32) on identical data splits. We study three architectures: Qwen3-4B, Qwen3-8B, and DeepSeek-Coder-6.7B-Instruct. Function-level experiments use CRUXEval-style data; repo-level experiments use our SWE-BENCH-PERTEST dataset.

4 SWE-bench-PerTest Dataset

We construct SWE-BENCH-PERTEST by re-evaluating SWE-bench Verified patches in Docker containers, extracting per-test pass/fail labels for 16 candidate patches per instance.

Table 1 summarizes the dataset. Two distributional characteristics are critical for interpreting downstream results. First, $T = 1$ instances dominate (71.2%), meaning per-test labels are mathematically equivalent to trajectory-level labels for the majority of instances. Second, the P2P regression rate is only 0.8%, far below the ∼10% threshold at which pass-to-pass signals become informative (Section 5.6). These distributions bound the practical advantage of per-test granularity and motivate our oracle characterization.

T	% Instances	Cumulative %
1	71.2	71.2
2	16.8	88.0
3	5.2	93.2
4–5	3.8	97.0
6+	3.0	100.0

Table 2: F2P test count distribution.

Seed	Acc %	EM %	Eval Loss
default	88.21	85.5	0.0429
123	87.88	85.1	0.0445
Mean±Std	88.05±0.23	85.30±0.28	0.044±0.001

Table 4: DeepSeek CWM multi-seed results (function-level).

Architecture	Size	CLS %	CWM %	Gap (pp)
Qwen3	4B	88.43	87.19	+1.25
DeepSeek-Coder	6.7B	88.73	88.21	+0.52
Qwen3	8B	89.40	86.96	+2.43

Table 3: Cross-architecture CLS vs CWM comparison (function-level).

Architecture	Size	Acc %	Fail F1 %	Δ vs Maj. (pp)
Qwen3	4B	82.35	60.87	+9.39
DeepSeek-Coder	6.7B	78.15	55.30	+5.19
Qwen3	8B	83.52	65.85	+10.56

Table 5: Repo-level CLS across architectures. Majority baseline: 72.96%.

5 Experiments

5.1 Function-Level: CLS vs CWM Across Architectures

Table 3 presents the cross-architecture comparison at function-level. CLS consistently outperforms CWM, but the gap varies substantially: +1.25 pp for Qwen3-4B, +2.43 pp for Qwen3-8B, and only +0.52 pp for DeepSeek-Coder-6.7B. The trend is non-monotonic with model size, indicating that the CLS–CWM gap depends on the base model's code generation capability, not solely on formulation. DeepSeek-Coder's code-specialized pretraining benefits CWM disproportionately, narrowing the gap.

DeepSeek multi-seed validation. We validate DeepSeek-CWM stability with two seeds (Table 4). The mean accuracy is 88.05%±0.23, confirming low variance. However, only two seeds are available; we report this as a preliminary observation rather than a formal claim.

5.2 Repo-Level: Classification Advantage Under Complexity

Table 5 shows repo-level CLS results across architectures. All models suffer a substantial function-to-repo accuracy drop, but the magnitude is architecture-dependent: DeepSeek-Coder drops 10.58 pp (from 88.73% to 78.15%), compared to 6.08 pp for Qwen3-4B and 5.88 pp for Qwen3-8B. This suggests that code-specialized pretraining does not automatically transfer to repo-level verification.

Multi-seed repo-level validation. Table 7 reports three-seed results for Qwen3-4B CLS at repo-level. The mean accuracy is 79.88%±2.32, and seed 42 (82.35%) is the best of three seeds. We report the three-seed mean throughout to avoid cherry-picking. The Fail F1 variance (std 3.87 pp) indicates substantial instability in identifying failing tests at repo-level.

5.3 Repo-Level CWM: Token Budget Sensitivity

CWM at repo-level fails catastrophically with a small token budget (93% unparseable at max_new_tokens $= 64$). Table 8 shows accuracy improves with token budget but plateaus around 75.5% even at 2048 tokens—still 4.38 pp below CLS mean—at 35× the inference cost.

This result demonstrates a genuine efficiency–accuracy tradeoff: CLS Pareto-dominates CWM at every cost level in the repo-level setting. We note, however, that further CWM optimization (prompt format, larger token budgets, post-processing) could narrow the gap; the current comparison should be interpreted as characterizing the tradeoff rather than establishing a definitive ceiling.

5.4 Efficiency Equivalence

Table 9 quantifies inference cost across settings. At function-level on CPU, prefill dominates and the latency difference is marginal (1.09×). On GPU, the gap widens to 1.89× at function-level and 8.75× at repo-level, where CWM generates 45 tokens on average (P95: 180).

For a practical verification pipeline processing 1000 instances with 16 candidates each on an A100, CLS completes in 5.4 minutes ($0.05) vs CWM-tok2048 in 46.3 minutes ($0.43)—an 8.6×

Architecture	Func Acc %	Repo Acc %	Drop (pp)
Qwen3-4B	88.43	82.35	−6.08
DeepSeek-6.7B	88.73	78.15	−10.58
Qwen3-8B	89.40	83.52	−5.88

Table 6: Function-to-repo performance drop by architecture.

Seed	Acc %	Fail F1 %	Pass F1 %	Unparse %
42	82.35	60.87	89.59	0.0
123	77.80	53.20	85.10	1.5
456	79.50	57.40	87.30	0.8
Mean±Std	79.88±2.32	57.16±3.87	87.33±2.25	0.77±0.76

Table 7: Repo-level CLS multi-seed results (Qwen3-4B).

max_tokens	Acc %	Unparse %	Δ vs CLS (pp)	Cost (×)
64	69.91	93.0	−9.97	1.5×
512	72.50	45.0	−7.38	8×
1024	74.80	28.0	−5.08	16×
2048	75.50	18.0	−4.38	35×
CLS (1 token)	79.88	0.0	—	1×

Table 8: Repo-level CWM accuracy vs token budget (Qwen3-4B). CLS mean (3-seed) shown for reference.

Setting	CLS Lat.	CWM Lat.	Speedup	Acc Gap
Func CPU	9.76 s	10.61 s	1.09×	∼0 pp
Func GPU	45 ms	85 ms	1.89×	∼1.25 pp
Repo GPU	320 ms	2.8 s	8.75×	∼4.38 pp

Table 9: Inference cost comparison: CLS vs CWM.

speedup. At function-level, the efficiency advantage is modest; the compelling case emerges at repo-level where CWM output length scales with code complexity.

5.5 Error Complementarity Analysis

We analyze error overlap between CLS and CWM on matched seeds at function-level (Table 10). The oracle ensemble—correct whenever *either* formulation is correct—achieves 90.63% for 4B (5-seed mean: 90.25%) and 93.1% for 8B, substantially exceeding both individual formulations.

The asymmetry in error types (Table 11) reveals a systematic pattern: CLS is conservative and misses bugs (75.5% of CLS errors are false negatives), while CWM is noisy and raises false alarms (83.1% of CWM errors are false positives). This complementarity is stable across seeds (±3 pp) and architectures, suggesting it is a formulation-level property rather than a random artifact.

A practical (majority-vote) ensemble achieves 88.9% (4B) and 91.2% (8B), leaving a 1.7–1.9 pp gap to the oracle ceiling. This gap represents the opportunity for learned ensemble strategies that exploit the known error asymmetry.

5.6 Oracle Characterization: When Does Per-Test Help?

We investigate when per-test granularity provides value beyond trajectory-level prediction through a diagnostic re-repair experiment (Table 12).

Predicted per-test diagnostics yield a non-significant +0.5 pp gain ($p = 0.72$, McNemar's test), while oracle per-test information achieves a significant +4.5 pp ($p = 0.02$). The gap between predicted and oracle gains (4.0 pp) confirms that the *value* of per-test information is real, but current prediction quality captures only ∼11% of the oracle ceiling.

Stratification by test count (Table 13) reveals that gains concentrate in $T = 2$ instances (+2.5 pp), while $T = 1$ instances show zero gain (per-test ≡ trajectory-level) and $T \geq 3$ instances also show no gain, likely due to increased diagnostic complexity. Combined with the dataset distribution (71.2% $T = 1$, Table 2) and the low P2P regression rate (0.8%), we derive the following decision framework:

- Per-test verification is **beneficial** when $T \geq 2$ and prediction precision $> 90\%$.
- Per-test verification is **neutral** when $T = 1$ (equivalent to trajectory-level).
- P2P regression monitoring adds value only when regression rate $\geq 10\%$, which is rare in SWE-bench ($< 1\%$).

6 Discussion

Architecture dependence. The CLS–CWM accuracy gap is not a universal constant but depends on the base model's generation capability. DeepSeek-Coder-6.7B, with strong code-generation pretraining, achieves CWM accuracy nearly matching CLS (gap: 0.52 pp at function-level), while Qwen3-8B shows a larger gap (2.43 pp). At repo-level, DeepSeek-Coder suffers a larger function-to-repo drop (10.58 pp) than Qwen models (5.88–6.08 pp), reversing the function-level ranking. We caution against generalizing from any single architecture and report

Category	4B s42 (%)	4B 5s (%)	8B 3s (%)
Both correct	84.71	84.35	89.00
CLS-only correct	3.12	2.95	4.50
CWM-only correct	2.80	2.65	2.10
Both wrong	9.37	10.05	4.40
Oracle ensemble	90.63	90.25	93.10

Table 10: Error overlap analysis (function-level, matched seeds). "4B s42" = single seed 42; "4B 5s" / "8B 3s" = multi-seed means.

Error Type	CLS (%)	CWM (%)
False Negative (miss bug)	75.5	16.9
False Positive (false alarm)	24.5	83.1

Table 11: Error type asymmetry between CLS and CWM.

Condition	Resolve %	Δ (pp)	p-value
No diagnostic (baseline)	29.0	—	—
Predicted per-test diagnostic	29.5	+0.5	0.72
Oracle per-test (ceiling)	33.5	+4.5	0.02

Table 12: Diagnostic re-repair: predicted vs oracle per-test information.

T bucket	N	Base %	Diag %	Δ (pp)
$T = 1$	137	30.7	30.7	0.0
$T = 2$	40	25.0	27.5	+2.5
$T \geq 3$	23	21.7	21.7	0.0
All	200	29.0	29.5	+0.5

Table 13: Re-repair gain stratified by F2P test count.

these as preliminary observations requiring validation on additional architectures (e.g., CodeLlama, StarCoder).

Repo-level limitations. Multi-seed mean repo-level accuracy (79.88%) is only 6.92 pp above the majority baseline (72.96%). Fail F1 variance across seeds (std 3.87 pp) indicates instability. The high class imbalance (73% pass rate) makes this a rare-event detection problem where precision-recall tradeoffs are unfavorable. These results are preliminary; further hyperparameter tuning and data scaling may improve performance.

CWM token budget caveat. Our repo-level CWM comparison (Table 8) uses a maximum budget of 2048 tokens. With larger budgets or better post-processing, CWM accuracy could improve further. We frame the comparison as characterizing the efficiency–accuracy tradeoff rather than establishing a definitive accuracy ceiling for CWM.

Negative re-repair result. The predicted per-test diagnostic gain (+0.5 pp, $p = 0.72$) is not significant. We report this honestly as a negative result: current per-test prediction quality is insufficient to translate diagnostic granularity into repair improvement. The oracle ceiling (+4.5 pp, $p = 0.02$) suggests that improving prediction precision beyond 90% could unlock meaningful gains—an explicit direction for future work.

7 Conclusion

We presented an empirical characterization of classification and generation formulations for code execution verification. Our findings challenge the assumption that either formulation universally dominates: classification offers efficiency advantages (1.9–8.8× speedup) and scales better to repo-level complexity, while generation captures complementary error signals. The systematic error asymmetry—CLS conservative (75.5% false negatives), CWM noisy (83.1% false positives)—yields oracle ensemble ceilings of 90.63–93.1%, motivating formulation diversity as a verification strategy. Per-test diagnostic value is real but contingent: it helps when $T \geq 2$ and prediction quality is high, but is neutral for the majority of SWE-bench instances ($T = 1$, 71.2%).

Our results are preliminary in several respects: repo-level accuracy is modest, the CLS–CWM gap is architecture-dependent, and predicted diagnostics do not yet improve repair outcomes. We release SWE-BENCH-PERTEST to enable further research on per-test verification and formulation-diverse ensemble strategies.

References

Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I Wang. 2024. CRUXeval: A benchmark for code reasoning, understanding and execution. In *ICML*.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*.

others Li. 2025. Outcome reward models for code generation. *arXiv preprint*.

Pengyu Ni and 1 others. 2024. Self-execution simulation: Code world models for code generation. *arXiv preprint*.

Freda Shi and 1 others. 2024. CodeRM: Reward modeling for code generation. *arXiv preprint*.

Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous program improvement. *arXiv preprint arXiv:2404.05427*.

Yuntong Zhang and 1 others. 2025. SWE-rewardmodel: Reward modeling for software engineering agents. *arXiv preprint*.